A Verified CPU/GPU Solver for the 3D Heat Equation

Karl Keshavarzi
Department of Electrical and Computer Engineering
University of Waterloo
github.com/karl-kes/3d-heat-solver

***Abstract*—We present a C++/CUDA solver for the 3D heat equation using explicit (forward Euler) finite differences, with CPU (OpenMP/SIMD) and CUDA GPU backends built from one source tree and selected at compile time. Both backends are verified independently against a closed-form analytic solution, a conservation law, an exact boundary-condition invariant, and a grid-convergence study. This caught a genuine GPU boundary-kernel defect and an invalid initial convergence-study design; after correction, the study recovers the expected second-order convergence rate. We report benchmarks from 8^3 to 512^3 cells on stated hardware. Nsight Compute measurements confirm the memory-bound conclusion directly, with a CPU/GPU crossover between 16^3 and 32^3 cells and a speedup plateau around 60–$68\times$.**

I. Introduction

The heat equation is a standard model problem for explicit time-stepping finite-difference methods, and a common first project for learning GPU programming: the update rule is a simple, highly parallel stencil, but a correct implementation still requires getting boundary conditions, stability, and memory layout right on both a CPU and a GPU backend. This paper presents and verifies such an implementation, with an emphasis on verifying correctness rather than only reporting a speedup number. A simulation that runs fast but computes the wrong answer is not useful; we treat the performance results in Section V as meaningful only because Section IV independently establishes that both backends solve the stated equation correctly.

The contributions of this work are:

- A dual-backend (OpenMP/SIMD CPU, CUDA GPU) solver for the 3D heat equation built from one source tree, with the backend selected at compile time.
- A verification methodology combining an analytic-solution check, a conservation check, an exact boundary-condition invariant, and a grid-convergence study (Section IV-D, including a corrected mesh-refinement scheme after an initial attempt was found to conflate resolution with problem difficulty, and an exact finite-domain eigenmode that isolates a boundary-modeling error in the Gaussian reference), applied independently to both backends.
- Identification and correction of a genuine GPU boundary-kernel defect that this verification methodology caught and a coarser check would have missed (Section III-D1).
- A performance characterization from 8^3 to 512^3 cells on stated hardware, with the memory-bound conclusion confirmed by direct profiler measurement rather than inference alone (Section V).

A. Related work

GPU finite-difference stencil computation is well studied; Micikevicius [1] describes shared-memory tiling and multi-GPU strategies for 3D finite-difference stencils on CUDA hardware, in the context of seismic modeling. That work targeted older Tesla-era GPUs, whose much smaller and less capable cache hierarchies made explicit shared-memory tiling necessary to capture neighbor reuse at all. The present work targets a single GPU and does not implement shared-memory tiling (Section VI); instead it relies on the much larger L1/L2 cache hierarchy of modern hardware (Ampere, here) to capture some of that reuse implicitly, consistent with the cache-reuse factor inferred in Section V.

II. Mathematical Formulation

A. Governing equation

We solve the unsteady heat equation on a cuboid domain,

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u, \tag{1}$$

where $u(x, y, z, t)$ is temperature and α is the (constant, isotropic) thermal diffusivity. The domain is discretized on a cell-centered structured grid with uniform spacings $\Delta x, \Delta y, \Delta z$ and one ghost-cell layer on each face; we write $n_x \times n_y \times n_z$ for the full per-axis cell count *including* those ghost layers, so an n-sized axis carries $n - 2$ interior cells. The Laplacian is approximated with the standard second-order central-difference 7-point stencil,

$$\nabla^2 u_{i,j,k} \approx \frac{u_{i-1,j,k} - 2u_{i,j,k} + u_{i+1,j,k}}{\Delta x^2} + (\text{similarly in } y, z). \tag{2}$$

B. Time integration and stability

Time advancement uses explicit (forward) Euler,

$$u^{t+1}_{i,j,k} = u^t_{i,j,k} + \alpha\, \Delta t\, \nabla^2 u^t_{i,j,k}. \tag{3}$$

This scheme is conditionally stable; von Neumann stability analysis [3] of the 3D case gives the classical Courant–Friedrichs–Lewy (CFL) limit [2]

$$\Delta t \leq \frac{1}{2\alpha\left(\Delta x^{-2} + \Delta y^{-2} + \Delta z^{-2}\right)}. \tag{4}$$

The implementation computes Δt from Eq. (4) with an 0.85 safety factor, so that changing the grid spacing or diffusivity from the command line cannot silently push the simulation into an unstable regime.

C. Boundary conditions

The domain boundary is insulated: zero heat flux normal to each face, $\partial u/\partial n = 0$. This is implemented with the standard ghost-cell mirroring technique, $u_{\text{ghost}} = u_{\text{adjacent interior}}$, applied independently on each of the six faces after every integration step. Because the central-difference stencil's flux term at the boundary depends only on the difference between the ghost and adjacent-interior values, this mirroring makes the discrete normal difference at the ghost interface exactly zero.

III. Implementation

A. Dual-backend architecture

Both the CPU and GPU code paths live in the same `.cu`/`.cuh` source files, selected at compile time via `#if defined(__CUDACC__)`. When no CUDA toolchain is available, CMake compiles the same sources as ordinary C++, and the project builds and runs with a pure CPU backend. This keeps the two implementations of each kernel textually adjacent, which made it straightforward to compare them directly when investigating the boundary-condition defect described in Section III-D1.

B. Memory layout

The field is stored in an aligned, struct-of-arrays container (`AlignedSoA`). Each dimension is padded up to a multiple of the SIMD width (16, 32, or 64 bytes, detected from `__AVX2__`/`__AVX512F__`) so that CPU stencil loops can be vectorized without unaligned loads, and so that the same padded layout is used unmodified on the GPU. Allocation is dispatched per backend: `cudaMalloc` under CUDA, `_aligned_malloc` on MSVC, and `std::aligned_alloc` elsewhere.

C. CPU parallelization

The CPU integration kernel parallelizes the two outer loop dimensions with `#pragma omp parallel for collapse(2)` and vectorizes the innermost loop with `#pragma omp simd` [5], relying on `__restrict__`-qualified pointers and an alignment hint (`__builtin_assume_aligned` on GCC/Clang, `__assume` on MSVC) so the compiler can generate aligned vector loads.

D. GPU kernel design

The integration kernel launches one thread per interior cell with a $8 \times 8 \times 8$ thread block [4], each thread computing one stencil update directly from global memory. The boundary condition is applied with three separate kernels, one per pair of opposing faces, each launched with a 2D thread grid sized to that face rather than to the full interior volume.

1) A correctness defect found by testing: The original boundary-condition kernel used a single 3D-launched kernel that derived all three face mirrors from one shared, interior-only-ranged thread index. Because that index never took the value 0 or $n-1$ in any dimension, edge cells where two boundary faces meet (e.g. $i = 0, j = 0$, any interior k) never received their second face's mirror update on the GPU, while the CPU implementation, which used a full range on the cross-axis in each face's loop, did not have this defect. The discrepancy is invisible to a coarse correctness check: a center-value comparison never samples an edge, and a total-energy check is not sensitive to a thin line of stale cells. It was caught by an explicit per-face boundary test (Section IV) and fixed by splitting the single kernel into three per-face kernels with correctly-sized 2D launches, mirroring the CPU code's loop ranges exactly. This also removed an efficiency problem in the original kernel, which launched $O(n^3)$ threads to perform $O(n^2)$ worth of boundary work.

IV. Verification

Following standard code-verification practice [6], we check the implementation against problems with a known closed-form solution rather than relying on benchmark performance as a proxy for correctness. Because the CPU and GPU backends are selected at compile time, a single binary cannot run both, so direct cross-binary comparison is not a natural fit. Instead, the same verification suite runs independently in both build configurations, comparing against either a closed-form analytic reference or a discrete invariant. For the analytic-reference checks, if both backends agree with the known-correct answer within a tolerance ϵ, the triangle inequality bounds their mutual disagreement by at most 2ϵ, without needing a dump-and-diff mechanism. Four checks are used: three on a small (32^3) grid with an isotropic Gaussian initial condition, and a fourth (Section IV-D) sweeping multiple grid sizes.

A. Analytic solution check

The initial condition is an isotropic Gaussian, $u(\mathbf{r}, 0) = A_0 \exp(-r^2/2\sigma_0^2)$. For the heat equation on an effectively unbounded domain, this remains Gaussian for all t, with

$$\sigma^2(t) = \sigma_0^2 + 2\alpha t, \qquad u(\mathbf{r}, t) = A_0 \left(\frac{\sigma_0^2}{\sigma^2(t)} \right)^{3/2} e^{-r^2/2\sigma^2(t)}. \tag{5}$$

The test runs the simulation for a fixed number of steps and compares the value at the grid point nearest the center against this closed form, within a 5% relative tolerance accounting for discretization error and the small residual influence of the (finite, insulated) domain boundary.

A Gaussian is exact only on an unbounded domain. As an exact reference for the *finite* insulated box we also use the Neumann cosine eigenmode

$$u(\mathbf{r}, t) = \left(\prod_{d \in \{x,y,z\}} \cos \frac{\pi r_d}{L_d} \right) e^{-\alpha \pi^2 t \sum_d 1/L_d^2}, \tag{6}$$

which satisfies $\partial u/\partial n = 0$ on every face exactly and so carries no boundary-modeling error. It drives the convergence study of Section IV-D.

B. Conservation of energy

With zero flux at every boundary and no source term, $\int u\,dV$ is exactly conserved: $\frac{d}{dt}\int u\,dV = \alpha \oint \nabla u \cdot \mathbf{n}\,dA = 0$. The test compares the discrete sum over interior cells only, $\sum_{i,j,k} u_{i,j,k}$, at $t = 0$ against the same sum after the run, within a 2% tolerance.

C. Exact boundary invariant

Because the boundary kernel implements $u_{\text{ghost}} = u_{\text{adjacent interior}}$ as a literal copy with no arithmetic, this is checked for *exact* equality (no tolerance) at every one of the six faces after the run. This is the test that caught the defect in Section III-D1.

D. Grid convergence

The checks above confirm the simulation is close to the analytic solution at one resolution; they do not confirm the scheme achieves its *designed* second-order spatial accuracy as the mesh is refined. We measured the global L_2 relative error against an analytic reference over $n \in \{16, 32, 64, 128, 256\}$, evaluated at every grid point rather than only the center (a single point can show artificially elevated order from symmetry-driven error cancellation).

A first attempt held $\Delta x = 1$ fixed and varied only n. This looked like mesh refinement but was not. The initial condition's width $\sigma_0 = 0.1n$ grew with n while the simulated time stayed fixed, so the Fourier number $\mathrm{Fo} = \alpha t/\sigma_0^2$ shrank as $1/n^2$ and the blob diffused less at each "finer" resolution. Each level was thus a progressively easier target rather than a real accuracy test, producing a false order of 3.5–5. Fixing the physical domain and σ_0 instead, and shrinking $\Delta x = L/n$, holds Fo approximately constant (0.14–0.17) across the sweep and gives the textbook order ≈ 2.04 over $n = 16$–128 against the Gaussian (Table I, Fig. 1); the order then degrades to 1.20 between $n = 128$ and 256.

That degradation is a property of the reference, not the scheme. The Gaussian is the infinite-domain heat kernel, so on the finite box it carries a fixed modeling error that does not shrink under refinement, and once the discretization error falls below it the observed order must drop. Repeating the sweep against the exact finite-domain eigenmode of Eq. (6) confirms this: it holds order ≈ 2.00 over $n = 16$–128 with no such degradation, isolating the Gaussian result as a boundary-modeling floor rather than time integration or precision. The eigenmode error at $n = 256$ instead reaches the single-precision floor ($\approx 10^{-5}$) and falls below the second-order trend, so its order is fit over $n = 16$–128. The CFL link $\Delta t \propto \Delta x^2$ quadruples the step count at each level (161 steps at $n = 128$, 641 at $n = 256$), which is why precision becomes visible at the finest grid.

Across all refinement levels, CPU and GPU solutions agreed to displayed precision at every grid point.

TABLE I
GLOBAL L_2 RELATIVE ERROR UNDER FIXED-DOMAIN REFINEMENT, AGAINST THE INFINITE-DOMAIN GAUSSIAN AND THE EXACT FINITE-DOMAIN COSINE EIGENMODE.

n	Gaussian	cosine
16	8.30×10^{-3}	2.96×10^{-3}
32	1.79×10^{-3}	7.41×10^{-4}
64	4.29×10^{-4}	1.84×10^{-4}
128	1.19×10^{-4}	4.62×10^{-5}
256	5.20×10^{-5}	5.74×10^{-6}

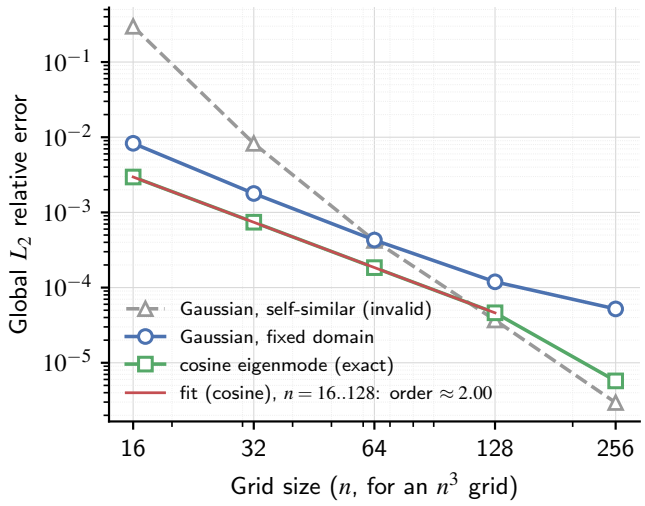

Fig. 1. Global L_2 relative error vs. grid size for the invalid self-similar sweep, the fixed-domain Gaussian sweep, and the exact cosine eigenmode, with the fitted order for the cosine. The Gaussian bends up at $n = 256$ (boundary-modeling floor); the cosine dips below the trend there (single-precision floor).

E. Continuous integration

A GitHub Actions pipeline runs the full test suite against the CPU backend on every push, since hosted runners have no GPU available to execute CUDA kernels. The GPU backend is compiled in CI to catch build breakage (`nvcc` does not require a physical device to compile, only to run), but its tests are run manually against real hardware.

V. Performance Evaluation

A. Experimental setup

All results were measured on: CPU, AMD Ryzen 5 8500G (6 cores / 12 threads); GPU, NVIDIA GeForce RTX 3070, 8 GB GDDR6 (256-bit bus, spec-sheet peak memory bandwidth 448 GB/s, peak FP32 throughput approximately 20.3 TFLOPS); 16 GB system RAM. Software: Windows 11; CUDA 13.3.33; MSVC 19.44.35228 (Visual Studio 2022, toolset 14.44); CMake 4.2.2 with Ninja. Both backends are built with `-O3` (CUDA and the non-MSVC host path) or `/O2` (MSVC host path) in Release configuration, with OpenMP enabled for the CPU backend. These results characterize this implementation on this specific desktop system and should not be read as an architecture-independent CPU/GPU comparison.

TABLE II
RUNTIME FOR 1000 FORWARD-EULER STEPS: MEAN ± SAMPLE STANDARD DEVIATION OVER 5 OF 6 RUNS (FIRST DISCARDED AS WARM-UP).

Grid size	CPU (ms)	GPU (ms)	Speedup
8^3	3.40 ± 0.70	25.5 ± 5.9	$0.13 \pm 0.04\times$
16^3	12.6 ± 0.53	34.4 ± 6.2	$0.37 \pm 0.07\times$
32^3	71.8 ± 1.4	26.5 ± 6.4	$2.71 \pm 0.66\times$
64^3	518 ± 23	28.4 ± 1.8	$18.2 \pm 1.4\times$
128^3	$3{,}768 \pm 50$	62.6 ± 0.6	$60.2 \pm 1.0\times$
256^3	$28{,}910 \pm 119$	444 ± 0.2	$65.1 \pm 0.3\times$
512^3	$226{,}716 \pm 900$	$3{,}339 \pm 1.0$	$67.9 \pm 0.3\times$

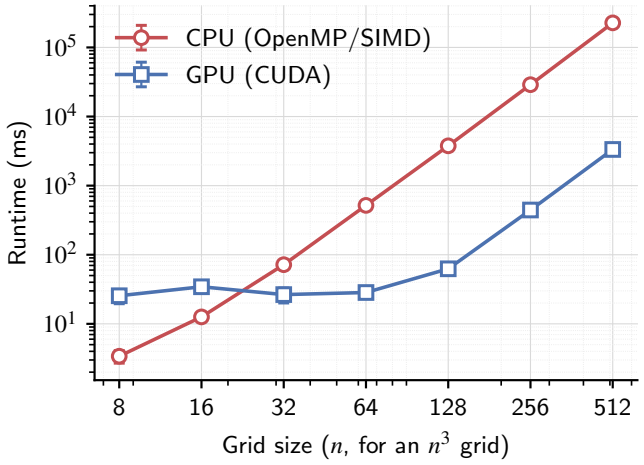

Fig. 2. Runtime vs. grid size for both backends (log-log).

Table II and Figs. 2–3 summarize benchmark results measured on the hardware described above. At 8^3 and 16^3 cells, fixed CUDA kernel-launch overhead (four launches per step: one integration kernel and three boundary kernels) exceeds the actual compute, and the CPU backend is faster. This is also why the GPU's relative run-to-run variance is largest in this small-grid range (18–24% for 8^3–32^3, Table II): runtime is dominated by launch overhead and timer granularity rather than by the stencil computation itself, and that variance drops sharply from 64^3 onward (to 1% or less by 128^3) once the actual computation dominates. The crossover occurs between 16^3 and 32^3 cells. Past the crossover, speedup grows quickly and then plateaus around 60–68× from 128^3 cells onward. This plateau coincides with achieved DRAM throughput approaching 90% of peak (Section V-B), indicating that further speedup is limited by memory bandwidth rather than available compute resources.

B. Bandwidth and occupancy measurements

The integration kernel's naive operational intensity [7], the ratio of floating-point operations to DRAM bytes moved assuming no cache reuse (as opposed to arithmetic intensity, which can include cache traffic), is fixed by the stencil: 16 floating-point operations and up to 8 float reads/writes (32 bytes) per cell update, giving OI ≈ 0.5 FLOP/byte. The RTX

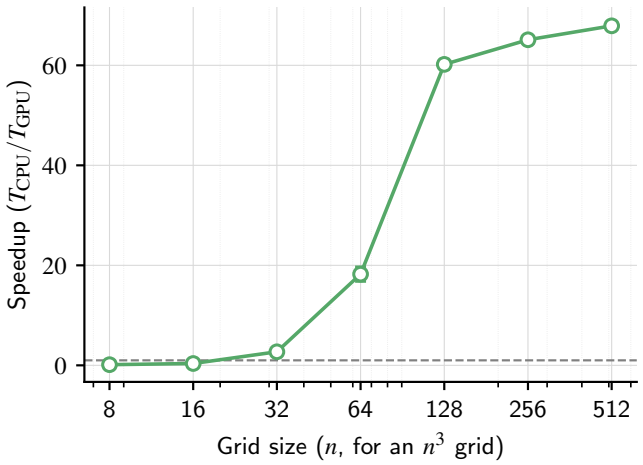

Fig. 3. Speedup (CPU time / GPU time) vs. grid size. The dashed line marks parity.

3070's roofline ridge point is at 20.3 TFLOPS/448 GB/s ≈ 45 FLOP/byte, two orders of magnitude above our OI, which places this kernel firmly in the bandwidth-bound region of the roofline model regardless of the achieved fraction of peak bandwidth. This naive intensity is only a lower bound. The measured L2 reuse below (75–85% hit rate) means the true DRAM traffic per cell falls below the no-reuse 32 bytes, so the actual DRAM operational intensity exceeds 0.5. Fig. 4 therefore plots each kernel at its *measured* operating point rather than the naive surrogate. The achieved FP32 rate is computed directly from the per-kernel duration as $16\,(n-2)^3/t_{\text{kernel}}$, and the achieved DRAM operational intensity as that rate divided by the measured DRAM bandwidth (Table IV). At the large grids whose working set exceeds the 4 MB L2 ($n \geq 128$), the achieved rate plateaus near 700 GFLOP/s at a DRAM operational intensity of ≈ 1.7–1.9 FLOP/byte. That intensity corresponds to ≈ 9 DRAM bytes per cell update, close to the compulsory minimum of 8: one 4-byte load of the current value and one 4-byte store of the updated result. It sits well above the naive 0.5 but still roughly 25× below the ridge point, so these points lie just under the bandwidth roof. The smaller grids ($n \leq 64$) are L2-resident or launch-overhead-dominated, so they fall well below the roof and do not represent the asymptotic regime.

We confirmed this directly with NVIDIA Nsight Compute 2026.2.0, profiling a steady-state launch of `gpuIntegrateGridKernel` (`--launch-skip 2 --launch-count 1`, skipping the first two launches to avoid driver/cache warm-up) across the full 8^3–512^3 range of Table II, averaged over 5 of 6 runs per size (first discarded as warm-up). Four metrics were collected individually (DRAM throughput, SM utilization, L2 hit rate, achieved occupancy; exact metric names in `scripts/profile.ps1`), not Nsight Compute's default full section set. Table III shows achieved DRAM throughput rising from under 1% of peak (under 4 GB/s) at 8^3 cells to 91.1% (408 GB/s) at 512^3. SM

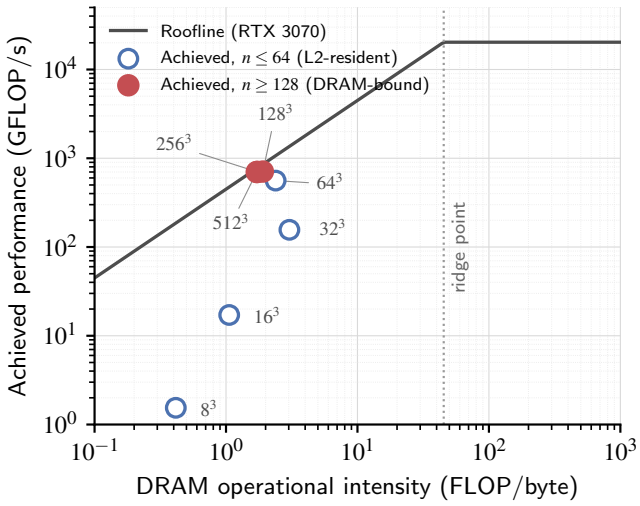

Fig. 4. Roofline model for the RTX 3070. Each grid size is plotted at its measured operating point (actual DRAM operational intensity, actual achieved FP32 rate). The large, DRAM-bound grids ($n \geq 128$) sit just under the bandwidth roof; the small, L2-resident grids ($n \leq 64$) fall well below it. The ridge point is marked for reference.

TABLE III
NSIGHT COMPUTE MEASUREMENTS OF `gpuIntegrateGridKernel`: MEAN ± SAMPLE STANDARD DEVIATION OVER 5 OF 6 RUNS PER SIZE (FIRST DISCARDED AS WARM-UP).

Grid size	DRAM %	GB/s	SM util. %	L2 hit %	Occupancy %
8^3	0.8 ± 0.4	3.7 ± 1.8	0.2 ± 0.0	85.1 ± 0.9	27.5 ± 0.3
16^3	3.6 ± 0.0	16.2 ± 0.1	1.9 ± 0.2	78.9 ± 1.3	29.9 ± 0.2
32^3	11.5 ± 0.3	51.5 ± 1.2	13.3 ± 0.1	76.5 ± 0.0	41.5 ± 0.1
64^3	52.4 ± 0.6	234.8 ± 2.5	43.2 ± 0.4	76.9 ± 0.1	76.9 ± 0.9
128^3	82.9 ± 0.2	371.5 ± 0.9	50.4 ± 0.1	78.4 ± 0.1	77.1 ± 0.3
256^3	90.6 ± 0.0	405.8 ± 0.2	45.6 ± 1.6	74.6 ± 0.5	79.6 ± 0.1
512^3	91.1 ± 0.3	408.2 ± 1.2	45.5 ± 0.2	74.6 ± 0.1	78.9 ± 0.3

utilization (the metric `sm__throughput.avg.pct_of_peak_sustained_elapsed`) stays in a narrow 43–50% band from 64^3 onward and never approaches saturation. This metric aggregates the SM's sub-units (instruction issue, load/store, and arithmetic pipelines) rather than FP32 throughput alone, so it counts the kernel's substantial address-arithmetic and load/store activity and overstates the fraction of FP32 peak actually reached. The true FP32 utilization, computed directly from the kernel duration (Table IV), is far lower, only $\approx 3.4\%$ of peak at the largest grids against the 91% of DRAM peak reached at the same sizes. The near-saturated DRAM throughput is therefore direct evidence that the kernel is memory-bandwidth-limited rather than compute-limited, not merely an inference from operational intensity (Table III, Fig. 5). At 8^3 and 16^3, both DRAM and SM utilization fall below 4% and occupancy drops to 28–30%, consistent with fixed kernel-launch overhead, not data movement, dominating runtime at those sizes. L2 hit rate stays in a 75–85% band across every size, consistent with substantial reuse of the stencil's overlapping neighbor reads.

TABLE IV
DERIVED KERNEL PERFORMANCE METRICS.

Grid size	Duration (μs)	GFLOP/s	FP32 %	OI_{DRAM}
8^3	2.24	1.5	0.01	0.41
16^3	2.56	19.6	0.10	1.21
32^3	2.76	156.2	0.77	3.04
64^3	6.81	560.0	2.76	2.39
128^3	45.2	708.1	3.49	1.91
256^3	373.7	701.7	3.46	1.73
512^3	3035.2	699.3	3.44	1.71

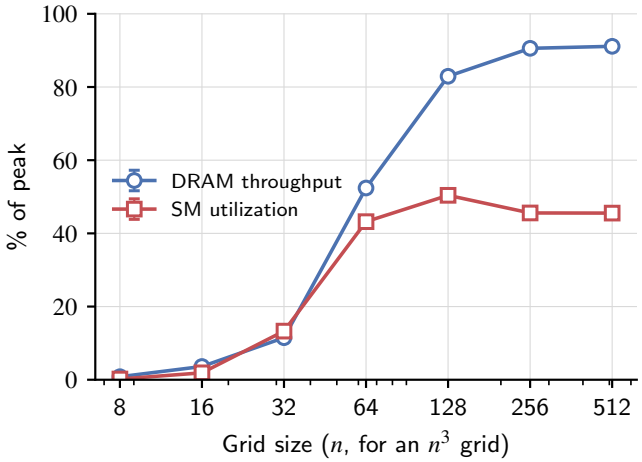

Fig. 5. Achieved DRAM throughput and SM utilization (% of peak) vs. grid size. DRAM throughput overtakes and pulls away from SM utilization beyond 64^3 cells.

VI. DISCUSSION AND LIMITATIONS

The solver targets a single GPU and a single explicit time-stepping scheme; it does not implement domain decomposition across multiple devices, shared-memory kernel tiling [1], or adaptive time stepping. Nor does it implement an implicit scheme such as Crank–Nicolson [8], which removes the CFL restriction at the cost of solving a linear system every step; the explicit scheme used here trades that restriction for a kernel with no inter-cell solve, which suits the goal of a straightforward CPU/GPU comparison. The verification strategy, checking both backends independently against a closed-form solution rather than diffing them against each other, depends on having a problem with a known analytic solution; it would not directly generalize to a production case with complex geometry or material heterogeneity without a different reference (e.g. a manufactured solution or a trusted reference implementation).

VII. CONCLUSION

Verifying both backends independently, rather than trusting a speedup number on its own, paid off twice in this work: it caught a real GPU boundary-kernel defect (Section III-D1), and it caught a flawed convergence-study design before that flaw could produce a misleadingly clean "order ≈ 4" result instead of the genuine order ≈ 2 (Section IV-D). With

those issues resolved, the GPU backend passes all verification checks (the analytic-solution, conservation, boundary-invariant, and convergence-order checks alike) and delivers a 60–$68\times$ speedup over the OpenMP/SIMD CPU backend at grid sizes of 128^3 and above, which profiler measurements show is consistent with the kernel's low operational intensity ($\approx$ 1.7 FLOP/byte measured at the DRAM-bound sizes, against a ridge point of 45): DRAM throughput reaches 91% of peak at 512^3 cells while the achieved FP32 rate is only $\approx 3.4\%$ of peak.

Artifact availability

Source code, build instructions, the verification and convergence test suite, and the benchmark and profiling scripts behind every result in this paper are publicly available at https://github.com/karl-kes/3d-heat-solver.

REFERENCES

[1] P. Micikevicius, "3D finite difference computation on GPUs using CUDA," in *Proceedings of the 2nd Workshop on General Purpose Processing on Graphics Processing Units (GPGPU-2)*, 2009.

[2] R. Courant, K. Friedrichs, and H. Lewy, "Über die partiellen Differenzengleichungen der mathematischen Physik," *Mathematische Annalen*, vol. 100, no. 1, 1928.

[3] G. G. O'Brien, M. A. Hyman, and S. Kaplan, "A study of the numerical solution of partial differential equations," *Journal of Mathematics and Physics*, vol. 29, no. 1, 1950.

[4] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2024.

[5] L. Dagum and R. Menon, "OpenMP: an industry standard API for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, 1998.

[6] W. L. Oberkampf and T. G. Trucano, "Verification and validation in computational fluid dynamics," *Progress in Aerospace Sciences*, vol. 38, no. 3, 2002.

[7] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, 2009.

[8] J. Crank and P. Nicolson, "A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 43, no. 1, 1947.